Estrutura de Dados

Curso Tecnologia em Análise de Desenvolvimento de Sistemas

Aula 06

Prof. Claudio Benossi

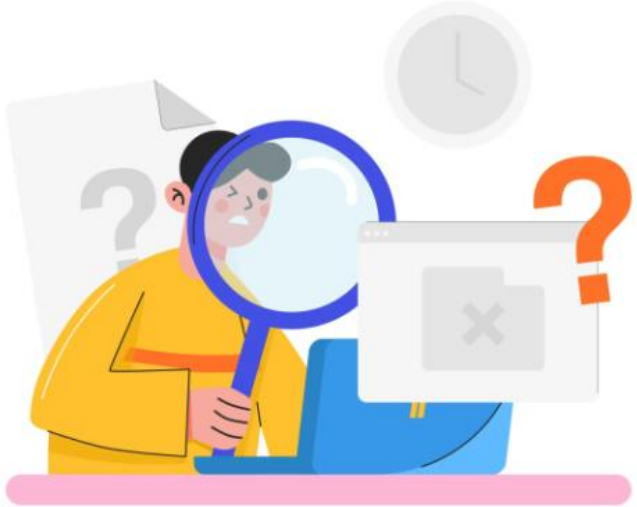
IV.
Unidade

Métodos elementares de ordenação

Métodos elementares de ordenação

Nesta aula veremos como aplicar algoritmos de ordenação em Java.

Estes algoritmos são utilizados em diversas necessidades em ocasiões específicas.



Métodos elementares de ordenação

Existem diversas aplicabilidades para a ordenação de dados, não só em Java, mas no mundo computacional como um todo.

A ideia de conhecer diversos algoritmos para ordenar dados é saber qual utilizar para conseguir um melhor desempenho ou uma melhor legibilidade de código, dependendo da necessidade.

Métodos elementares de ordenação

Não precisamos usar, por exemplo, um algoritmo “supercomplexo“ para ordenar 10 valores em um vetor, certo?

Seria desperdício de tempo desenvolver um algoritmo tão complexo para resolver um problema tão simples.

Métodos elementares de ordenação

Para analisar um algoritmo de ordenação você deve conhecer a sua complexidade no Pior Caso, Caso Médio e o melhor caso.

Métodos elementares de ordenação

Estas três características presentes em todos os algoritmos dizem respeito a sua velocidade dada uma situação específica.

Atente-se a esses conceitos, pois falaremos deles posteriormente, citando a complexidade do algoritmo através de **notação matemática**.

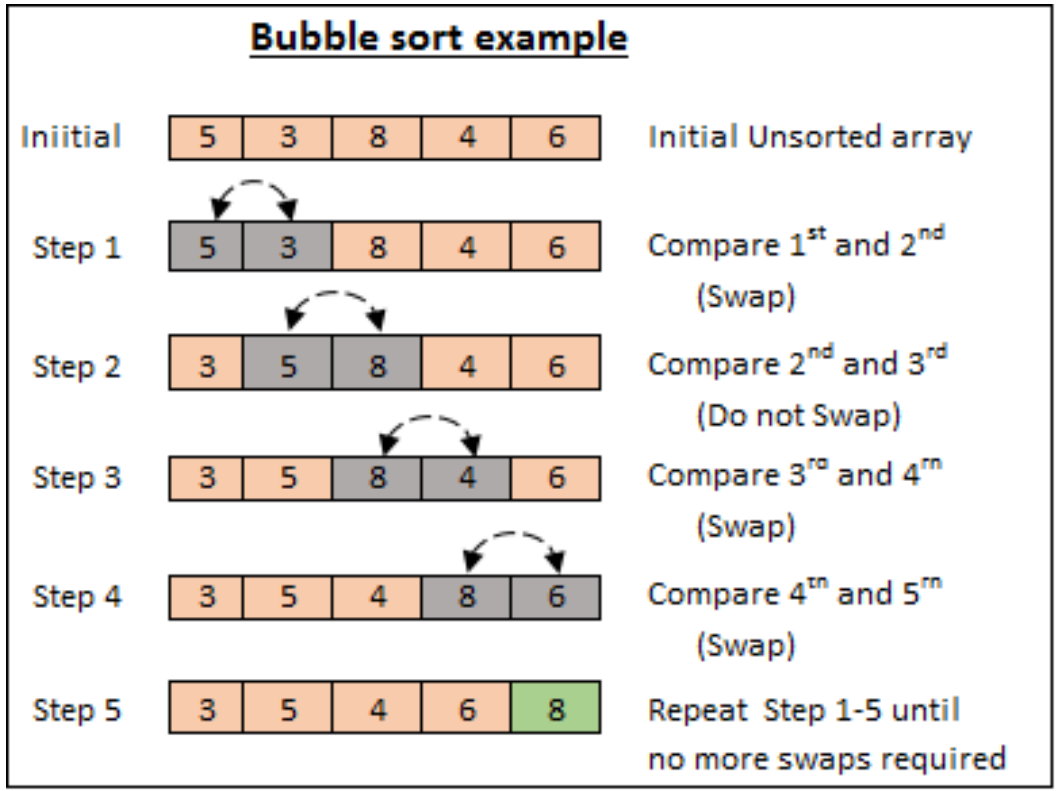
IV.
Unidade

Algoritmos de Ordenação

BubbleSort

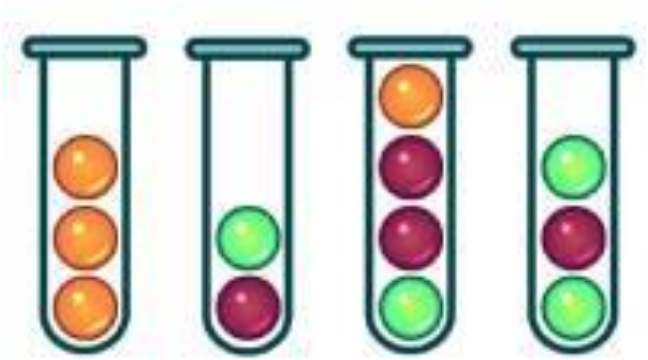
Algoritmos de Ordenação - BubbleSort

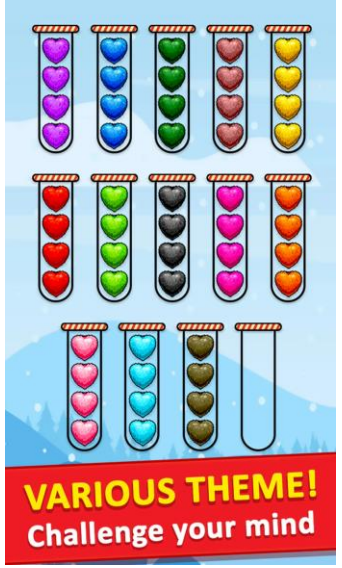
O BubbleSort é conhecido pela sua simplicidade e pela eficácia ao realizar ordenações em um número limitado de valores.



Algoritmos de Ordenação - BubbleSort

Seu princípio baseia-se na troca de valores entre posições consecutivas, fazendo com que valores altos ou baixos (dependendo da forma de ordenação desejada) “borbulhem” para o final da fila, por isso este algoritmo é chamado de BubbleSort.





Algoritmos de Ordenação - BubbleSort

Sua complexidade é:

Complexidade Pior Caso: $O(n^2)$

Complexidade Caso Médio: $O(n^2)$

Complexidade Melhor Caso: $O(n)$

Vimos que no melhor caso o seu tempo é quase inalterável, permanecendo constante, ou seja, um caso ideal.

Algoritmos de Ordenação - BubbleSort

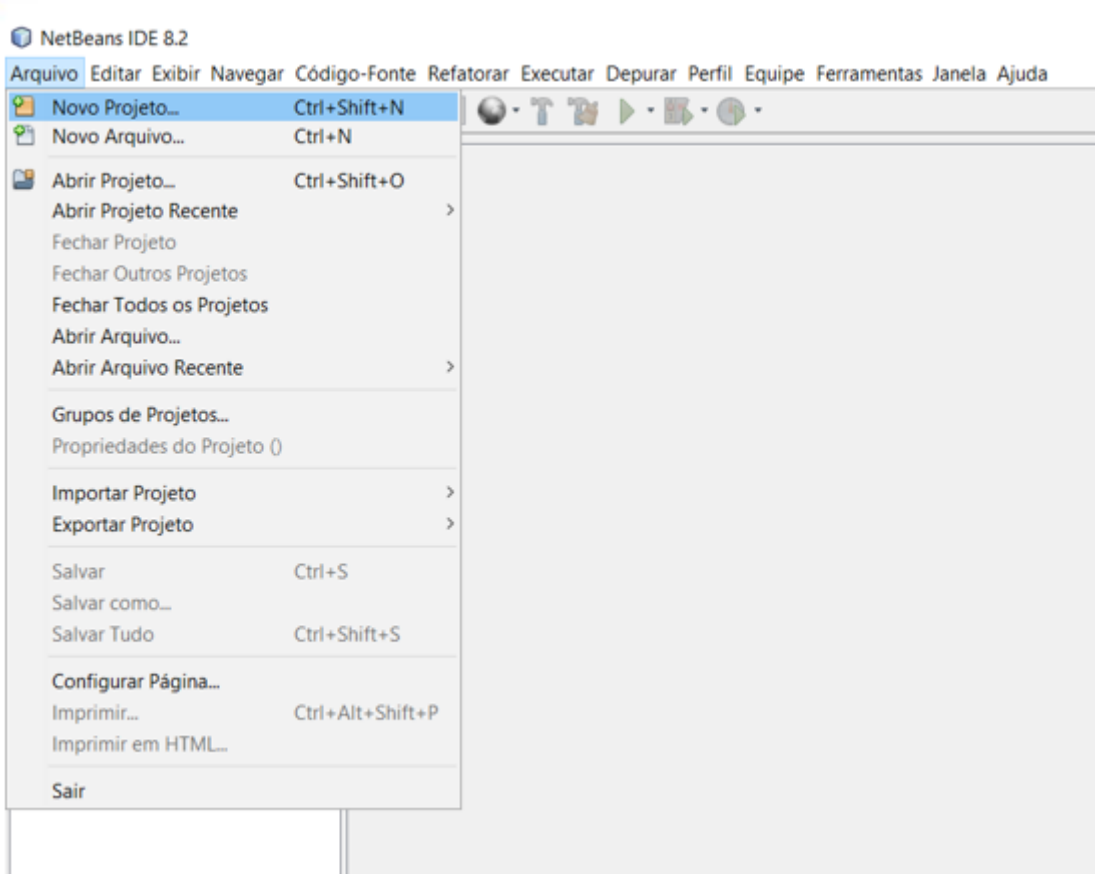
Vale lembrar que o algoritmo bubbleSorte não muito veloz, porém para pequenos conjuntos de dados ele é considerado muito bom.

Vejamos a implementação do bubbleSort na prática.

Para isso vamos criar uma aplicação em java com um array de números inteiros com 10 posições e inserir números de forma aleatória não ordenados.

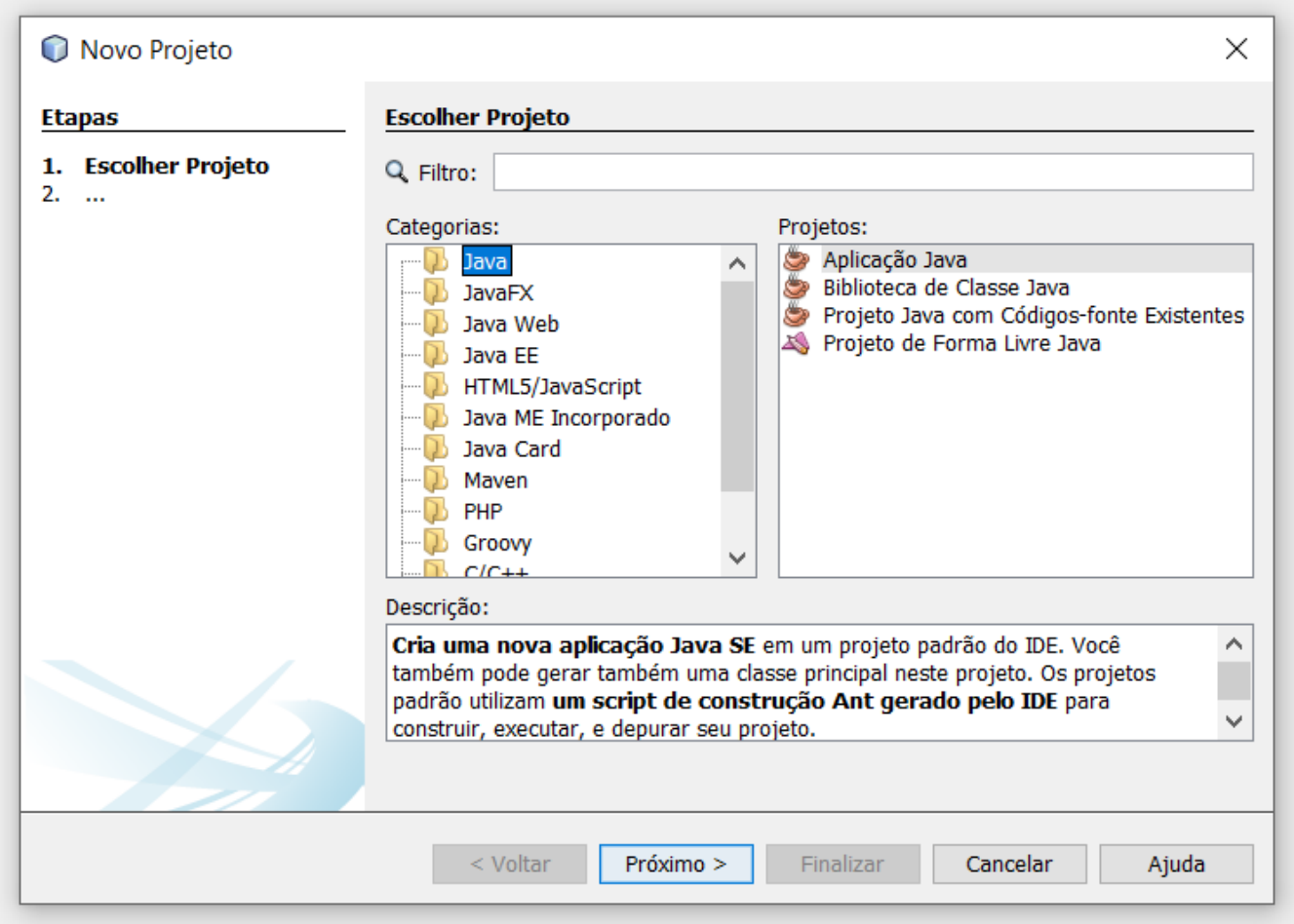
Algoritmos de Ordenação - BubbleSort

Vamos criar um novo Projeto



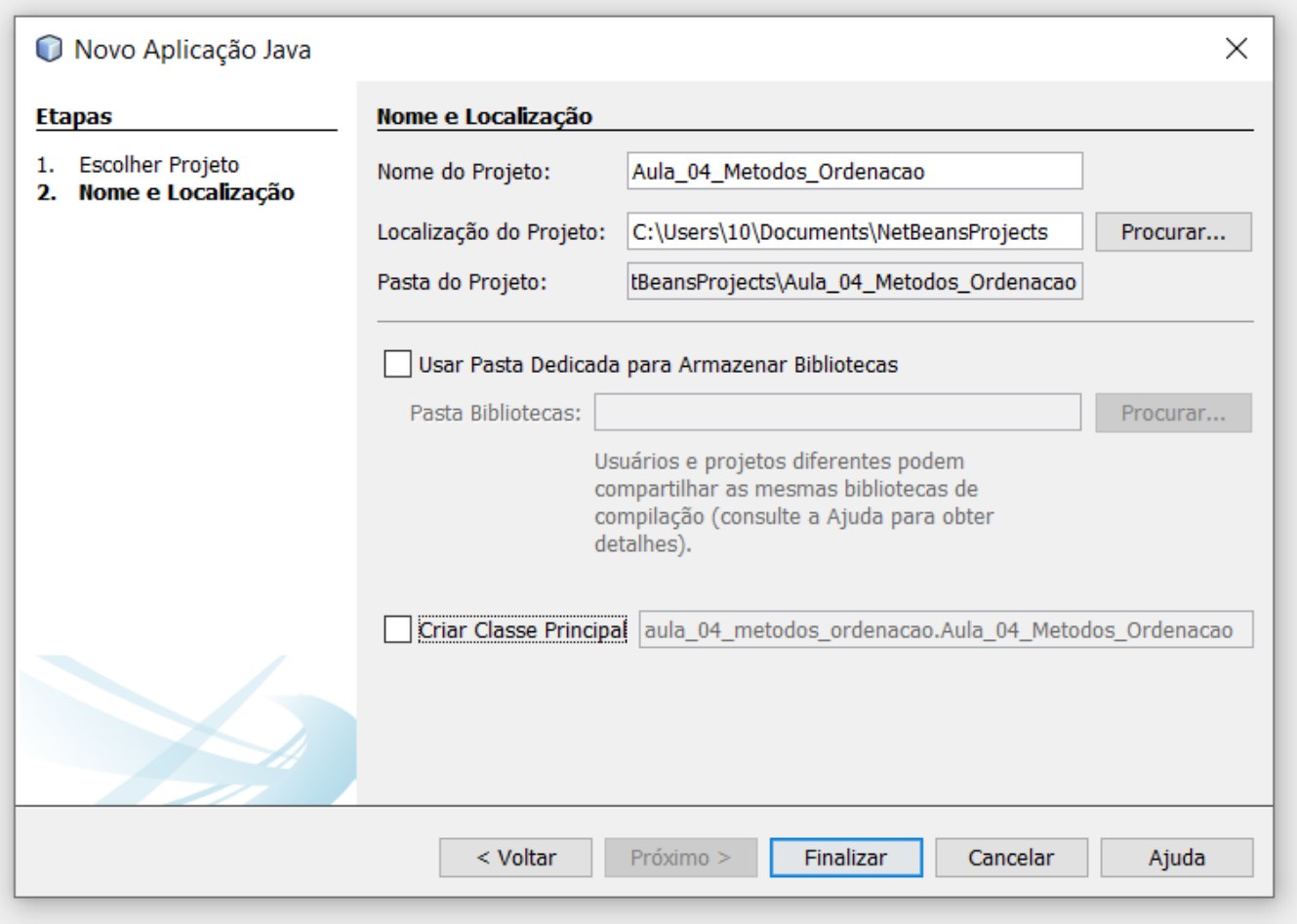
Algoritmos de Ordenação - BubbleSort

Selecione o tipo de Projeto:



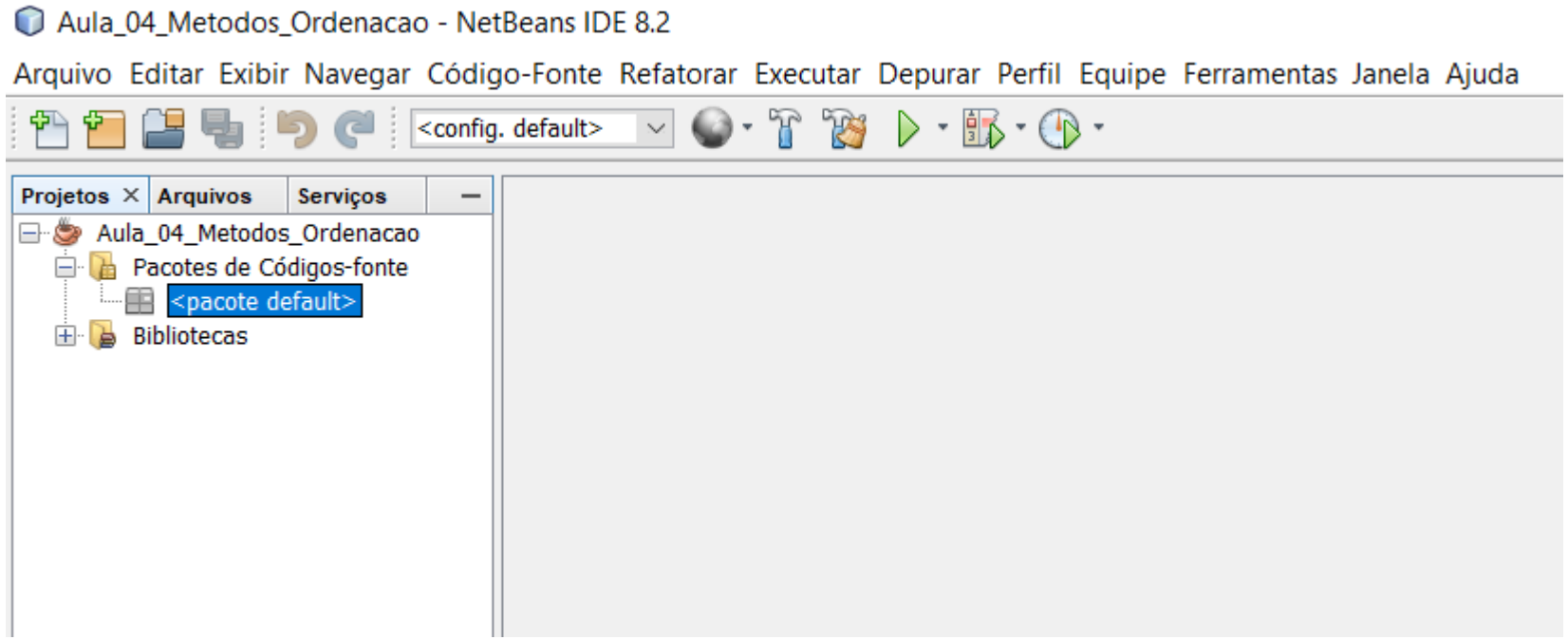
Algoritmos de Ordenação - BubbleSort

Informe o nome e a localização do seu projeto:



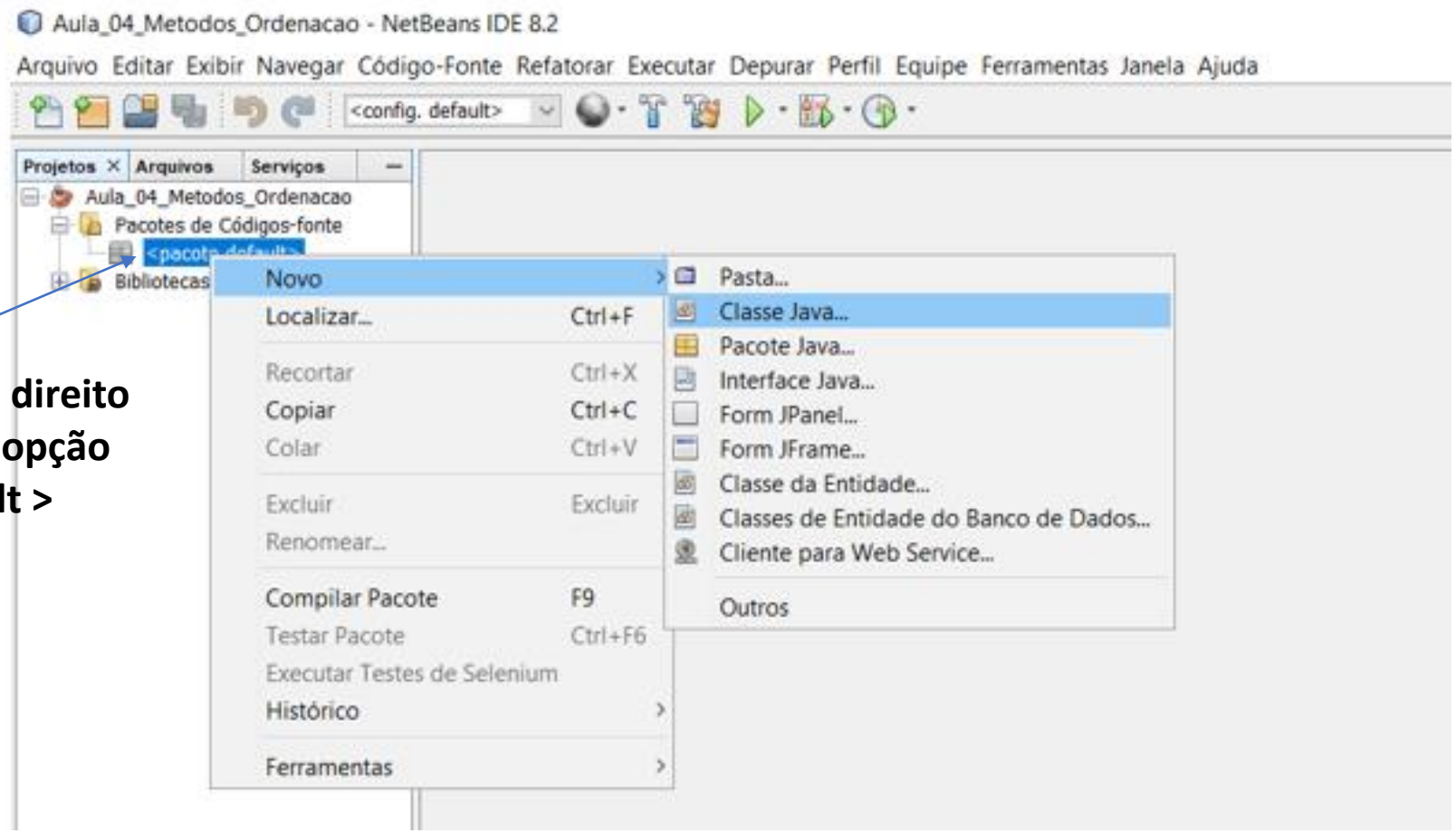
Algoritmos de Ordenação - BubbleSort

Pronto Projeto criado:



Algoritmos de Ordenação - BubbleSort

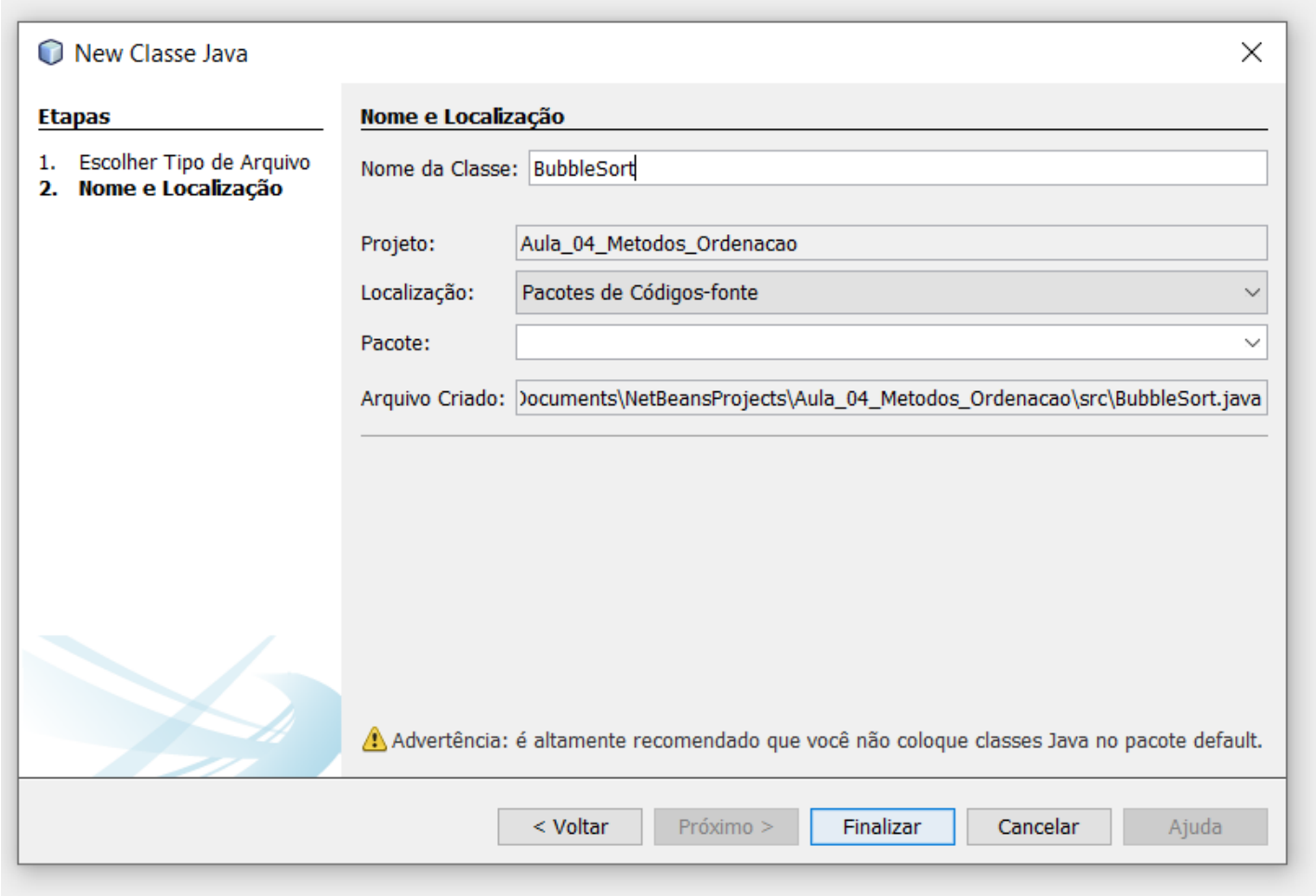
Vamos Criar uma classe chamada de BubbleSort:



Clique com o botão direito do mouse sobre a opção <pacote default >

Algoritmos de Ordenação - BubbleSort

Vamos Criar uma classe chamada de BubbleSort:



Algoritmos de Ordenação - BubbleSort

Vamos Criar uma classe chamada de BubbleSort:

BubbleSort.java

Código-Fonte Histórico

```java
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */

/**
 *
 * @author 10
 */
public class BubbleSort {

}
```

Algoritmos de Ordenação - BubbleSort

Vamos limpar os comentários e inserir um ponto de partida para a execução de nossa aplicação.

BubbleSort.java

Código-Fonte Histórico

```
1
2   public class BubbleSort {
3       public static void main(String[] args) {
4
5       }
6   }
7
```

Algoritmos de Ordenação - BubbleSort

Criando o vetor e inserindo os dados:

BubbleSort.java

Código-Fonte Histórico

```java

public class BubbleSort {
    public static void main(String[] args) {
        // criando um vetor de números inteiros de 10 posições
        int[] vetor = new int[10];

        // inserindo valores no vetor
        for(int i = 0; i < vetor.length; i++){
            vetor[i] = (int) (Math.random() * vetor.length);
        }
    }
}
```

Algoritmos de Ordenação - BubbleSort

Criando o vetor e inserindo os dados:

BubbleSort.java

Código-Fonte | Histórico

```java

public class BubbleSort {
    public static void main(String[] args) {
        // criando um vetor de números inteiros de 10 posições
        int[] vetor = new int[10];

        // inserindo valores no vetor
        for(int i = 0; i < vetor.length; i++){
            vetor[i] = (int) (Math.random() * vetor.length);
            // exibindo os valores do vetor
            System.out.println(vetor[i]);
        }
    }
}
```

Algoritmos de Ordenação - BubbleSort

Resultado:

Saída - Aula_04_Metodos_Ordenacao (run)

```
run:
2
6
7
1
8
7
6
5
5
0
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Observação:

Os valores apresentados podem ser diferentes, pois são criados de forma randomicamente.

Algoritmos de Ordenação - BubbleSort

Nosso vetor: aux=

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i j

O princípio do algoritmo BubbleSort, ele cria uma variável e fixa chamada de i em uma posição e cria uma segunda variável chamada de j para fazer a comparação para determinar qual é o maior valor e quando tem que fazer a troca de posição ele joga a informação em uma variável auxiliar que será chamada de aux.

Algoritmos de Ordenação - BubbleSort

Testando: aux=

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i j

Vetor no posição i é maior que Vetor no posição j ou (2 > 6)

Nesse caso não faz nada

Algoritmos de Ordenação - BubbleSort

Testando: aux=

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i j

Vetor no posição i é maior que Vetor no posição j ou (2 > 7)

Nesse caso não faz nada

Algoritmos de Ordenação - BubbleSort

Testando:

aux= 1

1	6	7	2	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i j

Vetor no posição i é maior que Vetor no posição j ou (2 > 1)

Nesse a variável aux recebe o valor do vetor[j];

o vetor[j] recebe o valor do vetor[i];

e o vetor[i] recebe a informação da variável aux;

Algoritmos de Ordenação - BubbleSort

Como era antes:

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

Como ficou:

1	6	7	2	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

Ainda não acabou ele continua...

Algoritmos de Ordenação - BubbleSort

Testando:

aux= 1

1	6	7	2	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i j

Vetor no posição i é maior que Vetor no posição j ou (1 > 8)

Nesse caso não faz nada

Algoritmos de Ordenação - BubbleSort

Testando:

aux= 1

1	6	7	2	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i j

Vetor no posição i é maior que Vetor no posição j ou (1 > 7)

Nesse caso não faz nada

Algoritmos de Ordenação - BubbleSort

Testando:

aux= 1

1	6	7	2	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i

j

Vetor no posição i é maior que Vetor no posição j ou (1 > 6)

Nesse caso não faz nada

Algoritmos de Ordenação - BubbleSort

Testando:

aux= 1

1	6	7	2	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i

j

Vetor no posição i é maior que Vetor no posição j ou (1 > 5)

Nesse caso não faz nada

Algoritmos de Ordenação - BubbleSort

Testando:

aux= 1

1	6	7	2	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

i

j

Vetor no posição i é maior que Vetor no posição j ou (1 > 5)

Nesse caso não faz nada

Algoritmos de Ordenação - BubbleSort

Testando:

aux= 0

0	6	7	2	8	7	6	5	5	1
0	1	2	3	4	5	6	7	8	9

i

j

Vetor no posição i é maior que Vetor no posição j ou (1 > 0)

Nesse a variável aux recebe o valor do vetor[j];

o vetor[j] recebe o valor do vetor[i];

e o vetor[i] recebe a informação da variável aux;

Algoritmos de Ordenação - BubbleSort

Como era antes:

1	6	7	2	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

Como ficou:

0	6	7	2	8	7	6	5	5	1
0	1	2	3	4	5	6	7	8	9

Ainda não acabou ele continua...

Algoritmos de Ordenação - BubbleSort

Testando: aux=

0	6	7	2	8	7	6	5	5	1
0	1	2	3	4	5	6	7	8	9
	i	j							

Vetor no posição i é maior que Vetor no posição j ou (6 > 7)

Nesse caso não faz nada

Algoritmos de Ordenação - BubbleSort

Testando: aux= 2

0	2	7	6	8	7	6	5	5	1
0	1	2	3	4	5	6	7	8	9

i j

Vetor no posição i é maior que Vetor no posição j ou (6 > 2)

Nesse a variável aux recebe o valor do vetor[j];

o vetor[j] recebe o valor do vetor[i];

e o vetor[i] recebe a informação da variável aux;

Algoritmos de Ordenação - BubbleSort

Como era antes:

0	6	7	2	8	7	6	5	5	1
0	1	2	3	4	5	6	7	8	9

Como ficou:

0	2	7	6	8	7	6	5	5	1
0	1	2	3	4	5	6	7	8	9

Ainda não acabou ele continua...

Algoritmos de Ordenação - BubbleSort

Testando:

aux= 2

0	2	7	6	8	7	6	5	5	1
0	1	2	3	4	5	6	7	8	9

i j

Vetor no posição i é maior que Vetor no posição j ou (2 > 8)

Nesse caso não faz nada

continua .... E continua ....

Algoritmos de Ordenação - BubbleSort

Voltando ao nosso código:

BubbleSort.java

Código-Fonte | Histórico

```
14            // Método de Ordenação BubbleSort
15            int aux;
16            for(int i = 0; i < vetor.length; i++){
17                for(int j = i + 1; j < vetor.length; j++){
18                    if(vetor[i] > vetor[j]){
19                        aux = vetor[j];
20                        vetor[j] = vetor[i];
21                        vetor[i] = aux;
22                    }
23                }
24            }
25
26        }
27    }
```

Algoritmos de Ordenação - BubbleSort

Voltando ao nosso código:

BubbleSort.java

Código-Fonte | Histórico

```
14            // Método de Ordenação BubbleSort
15            int aux;
16            for(int i = 0; i < vetor.length; i++){
17                for(int j = i + 1; j < vetor.length; j++){
18                    if(vetor[i] > vetor[j]){
19                        aux = vetor[j];
20                        vetor[j] = vetor[i];
21                        vetor[i] = aux;
22                    }
23                }
24            }
25
26        }
27    }
```

Algoritmos de Ordenação - BubbleSort

Agora vamos exibir o vetor ordenado:

BubbleSort.java

```
26            //Exibindo o nosso vetor ordenado
27            System.out.println("Nosso vetor Ordenado");
28            for(int i = 0; i < vetor.length; i++){
29                System.out.println(vetor[i]);
30            }
31
32        }
33    }
```

Algoritmos de Ordenação - BubbleSort

Testando:

Saída - Aula_04_Metodos_Ordenacao (run)

```
run:
6
6
4
7
3
8
2
6
1
8
Nosso vetor Ordenado
1
2
3
4
6
6
6
7
8
8
CONSTRUÍDO COM SUCESSO (tempo total: 1 segundo)
```

Algoritmos de Ordenação - BubbleSort

Bubble Sort é um algoritmo de ordenação que pode ser aplicado em Arrays e Listas dinâmicas.

Se o objetivo é ordenar os valores em forma decrescente, então, a posição atual é comparada com a próxima posição e, se a posição atual for maior que a posição posterior, é realizada a troca dos valores nessa posição.

Algoritmos de Ordenação - BubbleSort

Comentários sobre o Algoritmos de Ordenação BubbleSort:

- Grande quantidade de comparações;
- O BubbleSort é estável?
- Simples de entender e implementar.

Algoritmos de Ordenação - BubbleSort

Comentários sobre o Algoritmos de Ordenação BubbleSort:

- Uma desvantagem é que na prática ele tem execução lenta mesmo quando comparado a outros algoritmos quadráticos (n2).

Algoritmos de Ordenação - BubbleSort

Complexidade:

```
// Método de Ordenação BubbleSort
int aux;
for(int i = 0; i < vetor.length; i++){ // Complexidade O(n)
    for(int j = i + 1; j < vetor.length; j++){ // Complexidade O(n-1)
        if(vetor[i] > vetor[j]){
            aux = vetor[j];
            vetor[j] = vetor[i];
            vetor[i] = aux;
        }
    }
}
```

- Bubble Sort utiliza complexidade quadrática $O(n^2)$.

Algoritmos de Ordenação - BubbleSort

Agora para entender melhor a complexidade, vamos calcular o tempo de execução do nosso método de ordenação com diferentes quantidades armazenadas no nosso vetor.

BubbleSort.java | BubbleSortTime.java

Código-Fonte | Histórico

```java
public class BubbleSortTime {
    public static void main(String[] args) {
        int[] vetor = new int[100];

        for(int i = 0; i < vetor.length; i++){
            vetor[i] = (int) (Math.random() * vetor.length);
        }

       // variável que registra o tempo de execução no inicio
        long inicio = System.currentTimeMillis();

        // Método de Ordenação BubbleSort
        int aux;
        for(int i = 0; i < vetor.length; i++){
            for(int j = i + 1; j < vetor.length; j++){
                if(vetor[i] > vetor[j]){
                    aux = vetor[j];
                    vetor[j] = vetor[i];
                    vetor[i] = aux;
                }
            }
        }

        // Variável para registrar o tempo de execução no final
        long fim = System.currentTimeMillis();

        //Exibindo o Tempo de execução
        System.out.println("Tempo de execução> " + (fim-inicio));
    }
}
```

Saída - Aula_04_Metodos_Ordenacao (run)

```
run:
Tempo de execução> 1
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

BubbleSort.java | BubbleSortTime.java

Código-Fonte | Histórico

```java
public class BubbleSortTime {
    public static void main(String[] args) {
        int[] vetor = new int[10000];

        for(int i = 0; i < vetor.length; i++){
            vetor[i] = (int) (Math.random() * vetor.length);
        }

       // variável que registra o tempo de execução no inicio
        long inicio = System.currentTimeMillis();

        // Método de Ordenação BubbleSort
        int aux;
        for(int i = 0; i < vetor.length; i++){
            for(int j = i + 1; j < vetor.length; j++){
                if(vetor[i] > vetor[j]){
                    aux = vetor[j];
                    vetor[j] = vetor[i];
                    vetor[i] = aux;
                }
            }
        }

        // Variável para registrar o tempo de execução no final
        long fim = System.currentTimeMillis();

        //Exibindo o Tempo de execução
        System.out.println("Tempo de execução> " + (fim-inicio));
    }
}
```

Saída - Aula_04_Metodos_Ordenacao (run)

```
run:
Tempo de execução> 139
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

BubbleSort.java | BubbleSortTime.java

Código-Fonte | Histórico

```java
1  public class BubbleSortTime {
2      public static void main(String[] args) {
3          int[] vetor = new int[1000000];
4
5          for(int i = 0; i < vetor.length; i++){
6              vetor[i] = (int) (Math.random() * vetor.length);
7          }
8
9         // variável que registra o tempo de execução no inicio
10         long inicio = System.currentTimeMillis();
11
12         // Método de Ordenação BubbleSort
13         int aux;
14         for(int i = 0; i < vetor.length; i++){
15             for(int j = i + 1; j < vetor.length; j++){
16                 if(vetor[i] > vetor[j]){
17                     aux = vetor[j];
18                     vetor[j] = vetor[i];
19                     vetor[i] = aux;
20                 }
21             }
22         }
23
24         // Variável para registrar o tempo de execução no final
25         long fim = System.currentTimeMillis();
26
27         //Exibindo o Tempo de execução
28         System.out.println("Tempo de execução> " + (fim-inicio));
29     }
30 }
```

Saída - Aula_04_Metodos_Ordenacao (run)

```
run:
Tempo de execução> 1862089
CONSTRUÍDO COM SUCESSO (tempo total: 31 minutos 7 segundos)
```

Algoritmos de Ordenação - BubbleSort

BubbleSort		
Elementos	Caso médio (n^2)	Melhor Caso (n)
100	0 ms	0 ms
1.000	15 ms	0 ms
10.000	200 ms	0 ms
100.000	20.426 ms	2 ms
200.000	81.659 ms	4 ms

Algoritmos de Ordenação - BubbleSort

Exercício:

Faça um programa que receba 15 números via teclado e ordene esses números em ordem crescente usando o metodo Bubble Sort.

IV.
Unidade

Algoritmos de Ordenação

InsertionSort

Algoritmos de Ordenação - InsertionSort

Sua teoria baseia-se em ordenar os valores da esquerda para a direita, deixando os elementos lidos (a esquerda) ordenados.

Este é geralmente utilizado para ordenar um pequeno número de valores, onde faz-se muito eficiente.

Algoritmos de Ordenação - InsertionSort

A complexidade do código é:

Complexidade Pior Caso: $O(n^2)$

Complexidade Caso Médio: $O(n^2)$

Complexidade Melhor Caso: $O(n)$

Algoritmos de Ordenação - InsertionSort

Quando temos um caso onde a complexidade é n^2 devemos evitar, visto que a redução de desempenho deste algoritmo é exponencial.

Porém, no seu melhor caso temos uma constante n que significa a inalteração da velocidade, proporcional à quantidade de elementos.

Algoritmos de Ordenação - InsertionSort

Lembre-se que estamos trabalhando com proporcionalidade, então dizer que não varia não significa que um vetor de 10 elementos será ordenado na mesma velocidade de um vetor de um milhão de elementos, mesmo no melhor caso, porém a proporcionalidade entre a quantidade de elementos e sua velocidade continua constante, diferente do Pior Caso que aumenta ao quadrado.

Algoritmos de Ordenação - InsertionSort

O melhor caso ocorre quando todos os elementos já estão ordenados e o pior caso é exatamente o contrário, quando todos os elementos estão desordenados.

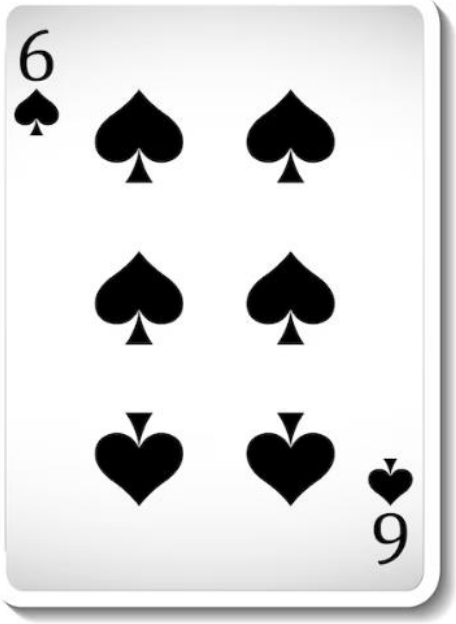
Algoritmos de Ordenação - InsertionSort

Para facilitar o entendimento, vamos imaginar um jogo de baralho.

Algoritmos de Ordenação - InsertionSort

Se no jogo você tem essas cartas e recebe uma nova sendo:

ou



Algoritmos de Ordenação - InsertionSort

Devemos comparar do final para o começo e vamos inserir a carta no local exato.

Algoritmos de Ordenação - InsertionSort

Nosso vetor: aux=

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

O princípio do algoritmo InsertionSort, ele cria uma variável chamada de i em uma posição e cria uma segunda variável chamada de j para fazer a comparação para determinar qual é o maior valor e quando tem que fazer a troca de posição ele joga a informação em uma variável auxiliar que será chamada de aux.

Algoritmos de Ordenação - InsertionSort

Nosso vetor:

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

Começando com a variável i sendo igual a 1 (percorrendo todo o vetor)
A variável aux é igual ao vetor na posição i (nesse caso 6)
A variável j é igual a variável i - 1 (nesse caso 1 – 1 = 0)
vamos fazer uma estrutura de repetição que enquanto a variável j for maior ou igual a zero e o vetor na posição j for maior que a variável aux; ele deve inserir no vetor na posição j + 1 o conteúdo do vetor na posição j e o valor da variável j passa a ser j – 1
Após sair dessa estrutura enquanto o vetor na posição j + 1 recebe o valor da variável aux.

Algoritmos de Ordenação - InsertionSort

Lógica:

```
VAR INT aux, j;
PARA (i=0; até que i < tamanho do vetor; PASSO 1)
        aux = vetor[i];
        j = i – 1;
        ENQUANTO (j >=0 e vetor[j] > aux)
                vetor[j + 1] = vetor[j];
        FIM ENQUANTO
        vetor[j + 1] = aux;
FIM PARA
```

Algoritmos de Ordenação - InsertionSort

Nosso vetor:

aux=6

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

j i

(0 >= 0) e (2 > 6)

Não faz nada

vetor[1] = 6

```
int aux, j;
for(int i = 1; i < vetor.length; i++){
    aux = vetor[i];
    j = i - 1;
    while(j >= 0 && vetor[j] > aux){
        vetor[j + 1] = vetor[j];
        j--;
    }
    vetor[j + 1] = aux;
}
```

Algoritmos de Ordenação - InsertionSort

Nosso vetor: aux=7

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

j i

(1 >= 0) e (6 > 7)

Não faz nada

vetor[2] = 7

```
int aux, j;
for(int i = 1; i < vetor.length; i++){
    aux = vetor[i];
    j = i - 1;
    while(j >= 0 && vetor[j] > aux){
        vetor[j + 1] = vetor[j];
        j--;
    }
    vetor[j + 1] = aux;
}
```

Algoritmos de Ordenação - InsertionSort

Vejamos um exemplo para entender melhor essa teoria.

Algoritmos de Ordenação - InsertionSort

Vamos criar uma classe InsertionSort, com um vetor e inserindo os dados:

BubbleSort.java | BubbleSortTime.java | InsertionSort.java

Código-Fonte | Histórico

```java
1   public class InsertionSort {
2       public static void main(String[] args) {
3           // criando um vetor de números inteiros de 10 posições
4           int[] vetor = new int[10];
5
6           // inserindo valores no vetor
7           for(int i = 0; i < vetor.length; i++){
8               vetor[i] = (int) (Math.random() * vetor.length);
9               // exibindo os valores do vetor
10              System.out.println(vetor[i]);
11          }
12
13      }
14  }
```

Algoritmos de Ordenação - InsertionSort

Vamos ordenar nosso vetor:

BubbleSort.java × BubbleSortTime.java × **InsertionSort.java** ×

Código-Fonte Histórico

```
13            // Método de Ordenação InsertionSort
14            int aux, j;
15            for(int i = 1; i < vetor.length; i++){
16                aux = vetor[i];
17                j = i - 1;
18                while(j >= 0 && vetor[j] > aux){
19                    vetor[j + 1] = vetor[j];
20                    j--;
21                }
22                vetor[j + 1] = aux;
23            }
24
25        }
26    }
```

Algoritmos de Ordenação - InsertionSort

Vamos Exibir os dados ordenados:

BubbleSort.java | BubbleSortTime.java | **InsertionSort.java**

Código-Fonte | Histórico

```
25            //Exibindo o nosso vetor ordenado
26            System.out.println("Nosso vetor Ordenado");
27            for(int i = 0; i < vetor.length; i++){
28                System.out.println(vetor[i]);
29            }
30        }
31    }
```

Algoritmos de Ordenação - InsertionSort

Vamos Testar:

Saída - Aula_04_Metodos_Ordenacao (run)

```
run:
3
4
6
4
1
9
3
4
8
2
Nosso vetor Ordenado
1
2
3
3
4
4
4
6
8
9
CONSTRUÍDO COM SUCESSO (tempo total: 1 segundo)
```

IV.
Unidade

Algoritmos de Ordenação

SelectionSort

Algoritmos de Ordenação - SelectionSort

No InsertionSort vimos uma ordenação simples, já no SelectionSort a implementação torna-se mais simples ainda, porém perdemos muito com o desempenho.

Este algoritmo tem por objetivo passar sempre o menor valor para a primeira posição (dependendo da ordem requerida pode ser o maior valor).

Algoritmos de Ordenação - SelectionSort

Então, para que isso seja feito ele percorre todos os elementos procurando um menor valor para só então colocá-lo na primeira posição, repetindo essa tarefa para cada um dos elementos.

Complexidade Pior Caso: $O(n^2)$

Complexidade Caso Médio: $O(n^2)$

Complexidade Melhor Caso: $O(n^2)$

Algoritmos de Ordenação - SelectionSort

Perceba que este algoritmo possui um péssimo desempenho, visto que sua complexidade é sempre exponencial, independente do caso em que estamos trabalhando.

Antes mesmo de mostrar qualquer código você já deve ser capaz de perceber que este algoritmo é bom para trabalhar-se com até, pelo menos, 10.000 elementos, dada a tabela do InsertionSort, visto sua simplicidade na implementação.

Algoritmos de Ordenação - SelectionSort

O SelectionSort guarda a posição do menor elemento na variável “menor” e percorre o array procurando por um valor menor. Caso este valor seja encontrado então a variável “menor” recebe a posição deste valor.

Por último é checado se a posição do menor elemento é diferente da posição atual, se isso for verdade então é feita uma troca de valores, colocando o menor elemento na frente.

Algoritmos de Ordenação - SelectionSort

Vamos entender o passo a passo do algoritmo apresentado: começamos com o laço for, que percorre todos os elementos, guardando o valor da iteração atual na variável “fixo”.

Em seguida usamos uma variável auxiliar chamada menor que guardará o valor de fixo.

Algoritmos de Ordenação - SelectionSort

Tendo em mãos a variável menor, que na primeira iteração é igual ao primeiro elemento do vetor, percorremos (através de outro laço for) todos os elementos do array partindo do segundo elemento até o último.

Algoritmos de Ordenação - SelectionSort

Então imagine a situação, se temos o array com as posições 0,1,2,3 a variável menor guardará o valor '0' e o segundo laço irá percorrer os elementos na posição 1,2,3.

Algoritmos de Ordenação - SelectionSort

A cada vez que um elemento for visitado, no segundo laço for, é feita uma comparação se o elemento atual é menor que o elemento que está na posição indicada pela variável menor.

Se isso for verdade então a variável menor recebe o valor de i, ou seja, a nova posição do menor elemento será o atual lido.

Algoritmos de Ordenação - SelectionSort

Se a variável menor guarda a posição do menor elemento, então sempre que um elemento menor que o menor atual for encontrado, a posição do menor será alterada para o atual.

Algoritmos de Ordenação - SelectionSort

No final do segundo laço for teremos na variável menor a posição do menor elemento lido, se esta posição for diferente da posição atual que está na variável fixo então faremos uma troca.

Algoritmos de Ordenação - SelectionSort

Lembre-se que a variável fixo guarda a posição do elemento que está sendo lido no primeiro laço for, e a variável menor guarda a posição do menor elemento encontrado em todo o vetor.

Sendo assim, se ambos forem diferentes capturamos o valor contido na posição da variável “fixo” e armazenamos na variável “t”, apenas para não perder o valor.

Algoritmos de Ordenação - SelectionSort

Depois colocamos o valor do elemento na posição da variável “menor” dentro da posição da variável “fixo”.

O que temos até agora é que o menor elemento está na primeira posição do vetor e também em outra posição, ou seja, na sua original, agora precisamos colocar o valor de “t” dentro da posição antiga do menor elemento.

Algoritmos de Ordenação - SelectionSort

SelectionSort		
Elementos	Caso médio (n^2)	Melhor Caso (n)
100	0 ms	0 ms
1.000	12 ms	10 ms
10.000	67 ms	62 ms
100.000	5110 ms	4561 ms
200.000	20.435 ms	18.181 ms

Algoritmos de Ordenação - SelectionSort

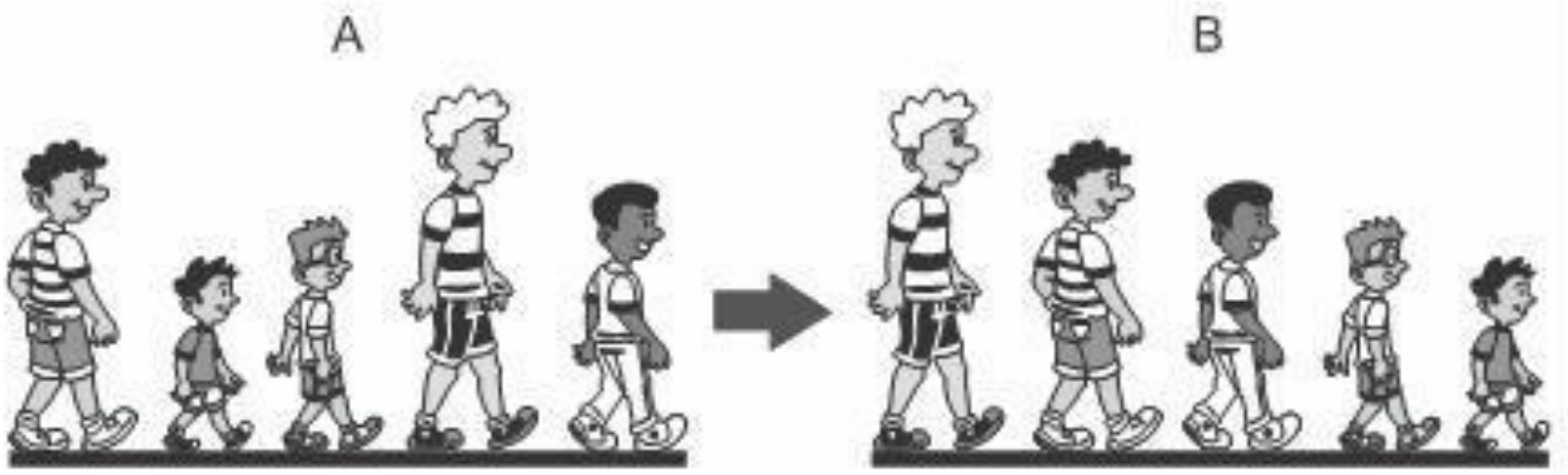
Para facilitar o entendimento vamos pensar em uma professora organizando a fila dos alunos

Algoritmos de Ordenação - SelectionSort

Este algoritmo procura o menor elemento e insere na posição adequada, do menor para o maior.

Algoritmos de Ordenação - SelectionSort

Obs. Também podemos modificar o algoritmo para colocar do maior para o menor.



Algoritmos de Ordenação - SelectionSort

Nosso vetor: aux=

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

Vamos pergunta de o vetor na posição 0 é menor que os demais, quando achar, armazena o valor na auxiliar, realiza a troca.

Algoritmos de Ordenação - SelectionSort

Vamos observar o código:

BubbleSort.java | BubbleSortTime.java | InsertionSort.java | **SelectionSort.java**

Código-Fonte | Histórico

```java
1   public class SelectionSort {
2       public static void main(String[] args) {
3           // criando um vetor de números inteiros de 10 posições
4           int[] vetor = new int[10];
5
6           // inserindo valores no vetor
7           for(int i = 0; i < vetor.length; i++){
8               vetor[i] = (int) (Math.random() * vetor.length);
9               // exibindo os valores do vetor
10              System.out.println(vetor[i]);
11          }
12      }
13  }
```

Algoritmos de Ordenação - SelectionSort

Vamos observar o código:

BubbleSort.java | BubbleSortTime.java | InsertionSort.java | **SelectionSort.java**

Código-Fonte | Histórico

```
12
13          // Método de Ordenação SelectionSort
14          int menor_posicao, aux;
15          for(int i = 0; i < vetor.length; i++){
16              menor_posicao = i;
17              for(int j = i + 1; j < vetor.length; j++){
18                  if(vetor[j] < vetor[menor_posicao]){
19                      menor_posicao = j;
20                  }
21              }
22              aux = vetor[menor_posicao];
23              vetor[menor_posicao] = vetor[i];
24              vetor[i] = aux;
25          }
26      }
27  }
```

Algoritmos de Ordenação - SelectionSort

Vamos observar o código:

BubbleSort.java | BubbleSortTime.java | InsertionSort.java | **SelectionSort.java**

Código-Fonte | Histórico

```
26
27            //Exibindo o nosso vetor ordenado
28            System.out.println("Nosso vetor Ordenado");
29            for(int i = 0; i < vetor.length; i++){
30                System.out.println(vetor[i]);
31            }
32        }
33    }
34
```

Algoritmos de Ordenação - SelectionSort

Vamos testar:

Saída - Aula_04_Metodos_Ordenacao (run)

```
run:
5
1
0
7
0
5
9
9
8
5
Nosso vetor Ordenado
0
0
1
5
5
5
7
8
9
9
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

IV.
Unidade

Algoritmos de Ordenação

QuickSort

Algoritmos de Ordenação - QuickSort

Este algoritmo usa uma técnica conhecida por divisão e conquista, muito conhecida por reduzir problemas complexos em problemas menores para tentar chegar em uma solução.

Sendo assim, o resultado do problema inicial é dada como a soma do resultado de todos os problemas menores.

Algoritmos de Ordenação - QuickSort

Sua complexidade é:

Complexidade Pior Caso: $O(n^2)$

Complexidade Caso Médio: (nlogn)

Complexidade Melhor Caso: (nlogn)

Algoritmos de Ordenação - QuickSort

O QuickSort sai na frente de outros algoritmos mais simples quando tratamos do caso médio, por trabalhar com logaritmo (nlogn), o que torna o resultado mais rápido do que o InsertionSort e o QuickSort.

Algoritmos de Ordenação - QuickSort

O algoritmo consiste nos seguintes passos:

1º - Escolhe-se um elemento qualquer da lista, que será chamado de pivô.

2º - Todos os elementos antes do pivô devem ser menores que ele e todos os elementos após o pivô devem ser maiores que ele. Quando esse processo de separação for finalizado, então o pivô deve estar exatamente no meio da lista.

3º - De forma recursiva os elementos da sublista menor e maiores são ordenados.

Algoritmos de Ordenação - QuickSort

Nosso vetor:

2	6	7	1	8	7	6	5	5	0
0	1	2	3	4	5	6	7	8	9

pivô

Algoritmos de Ordenação - QuickSort

O QuickSort é um algoritmo mais robusto e complexo que os mostrados anteriormente, por isso achamos importante exemplificar o seu uso através do pseudocódigo.

Pode até ser visto como um dos mais rápidos, dependendo da situação.

Algoritmos de Ordenação - QuickSort

QuickSort	
Elementos	Caso médio (nlogn)
100	0 ms
1.000	0 ms
10.000	39 ms
100.000	43 ms
200.000	50 ms

Algoritmos de Ordenação - QuickSort

Vamos analisar o nosso código:

Criamos o vetor e exibimos os dados do vetor

BubbleSort.java | BubbleSortTime.java | InsertionSort.java | SelectionSort.java | QuickSort.java

Código-Fonte | Histórico

```java
1   public class QuickSort {
2       public static void main(String[] args) {
3           // criando um vetor de números inteiros de 10 posições
4           int[] vetor = new int[10];
5
6           // inserindo valores no vetor
7           for(int i = 0; i < vetor.length; i++){
8               vetor[i] = (int) (Math.random() * vetor.length);
9               // exibindo os valores do vetor
10              System.out.println(vetor[i]);
11          }
12      }
13  }
```

Algoritmos de Ordenação - QuickSort

Como esse método de ordenação ele cria uma partição e pode ser recursivo (que pode ser repetido, ou ainda um código que chama ele mesmo), vamos criar um método para indicar o pivô e outro chamado de ordenaQuickSort.

BubbleSort.java | BubbleSortTime.java | InsertionSort.java | **SelectionSort.java** | **QuickSort.java**

Código-Fonte | Histórico

```java
14        // método para ordenar
15        static void ordenaQuickSort(int[] vetor, int esquerda, int direita){
16            if(esquerda < direita){
17                int p = particao(vetor, esquerda, direita);
18                ordenaQuickSort(vetor, esquerda, p);
19                ordenaQuickSort(vetor, p + 1, direita);
20            }
21        }
22
23        // método para criar as partições (dividir o vetor em partes)
24        static int particao(int[] vetor, int esquerda, int direita){
25            int meio = (int) (esquerda + direita) / 2;
26            int pivo = vetor[meio];
27            int i = esquerda - 1;
28            int j = direita + 1;
29            while(true){
30                do{
31                    i++;
32                } while(vetor[i] < pivo);
33                do{
34                    j--;
35                } while(vetor[j] > pivo);
36                if(i >= j){
37                    return j;
38                }
39                int aux = vetor[i];
40                vetor[i] = vetor[j];
41                vetor[j] = aux;
42            }
43        }
44    }
```

Algoritmos de Ordenação - QuickSort

Agora vamos voltar ao nosso código e inserir a chamada do método para realizar a ordenação e também um comando para exibir o vetor agora ordenado.

BubbleSort.java | BubbleSortTime.java | InsertionSort.java | SelectionSort.java | QuickSort.java

Código-Fonte | Histórico

```java
public class QuickSort {
    public static void main(String[] args) {
        // criando um vetor de números inteiros de 10 posições
        int[] vetor = new int[10];

        // inserindo valores no vetor
        for(int i = 0; i < vetor.length; i++){
            vetor[i] = (int) (Math.random() * vetor.length);
            // exibindo os valores do vetor
            System.out.println(vetor[i]);
        }

        // chamando o método para a ordenação
        ordenaQuickSort(vetor, 0, vetor.length - 1);

        //Exibindo o nosso vetor ordenado
        System.out.println("Nosso vetor Ordenado");
        for(int i = 0; i < vetor.length; i++){
            System.out.println(vetor[i]);
        }
    }

    // método para ordenar
    static void ordenaQuickSort(int[] vetor, int esquerda, int direita){
        if(esquerda < direita){
            int p = particao(vetor, esquerda, direita);
            ordenaQuickSort(vetor, esquerda, p);
            ordenaQuickSort(vetor, p + 1, direita);
        }
    }
```

Algoritmos de Ordenação - QuickSort

Vamos testar:

Saída - Aula_04_Metodos_Ordenacao (run)

```
run:
5
9
2
5
2
4
3
4
6
4
Nosso vetor Ordenado
2
2
3
4
4
4
5
5
6
9
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Algoritmos de Ordenação

OK

Algoritmos de Ordenação

Vamos criar uma tabela de comparação de tempo de execução entre os métodos apresentados e fazer uma análise rápida sobre o tema:

Método	100 elementos	1.000 elementos	10.000 elementos	100.000 elementos
BubbleSort				
InsertionSort				
SelectionSort				
QuickSort				

Milissegundos, onde 1 segundo é igual a 1.000 Milissegundos

"Quando a gente acha que tem todas as respostas,
vem a vida e muda todas as perguntas”

Luís Fernando Veríssimo

Obrigado!

Se precisar ...

Prof. Claudio Benossi

claudio.benossi@sp.senac.br